



Reinforcement learning path planning with A*-Initialized DDQN in dynamic and partially observable environments

Zimin Xu^a, Guangcheng Pang^a, Youyong Lai^a, Yugan Huang^a, Jianlei Zhang^{b,*}

^a School of Computer, Electronics and Information, Guangxi University, Nanning, 530004, PR China

^b College of Artificial Intelligence, Nankai University, Tianjin, 300350, PR China

HIGHLIGHTS

- Populating the replay buffer with A* boosts early training efficiency and convergence.
- Custom action strategy improves path safety and decision-making during exploration.
- The reward function ensures safe, efficient and smooth path planning in complex maps.

ARTICLE INFO

Keywords:

Path planning
Reinforcement learning
Double deep Q-network
Experience replay buffer
Action selection mechanism

ABSTRACT

Path planning algorithms play a critical role in various domains, yet navigating safe paths in unknown and complex environments using only local perception remains a substantial challenge. This paper presents A*-IDDQN, a novel path planning algorithm that integrates an improved Double Deep Q-Network (DDQN) with the A* algorithm. To address the challenge of extensive exploration during the initial training phase of Reinforcement Learning (RL), we initialize the DDQN's experience replay buffer with paths generated by A*. This initialization allows the agent to quickly become familiar with the environment, reducing reliance on random exploration and significantly enhancing training efficiency. To optimize the trade-off between exploration and exploitation, we propose an enhanced ϵ -greedy strategy, supplemented by an additional action selection mechanism, providing greater flexibility in decision-making. The reward function is carefully designed to prevent local optima, incorporating four key terms: target-reaching reward, obstacle avoidance penalty, proximity reward, and turning penalty. These elements work in tandem to ensure path safety, efficiency, and smoothness. Experimental results demonstrate that A*-IDDQN consistently achieves high success rates in path planning, exhibits stable training dynamics, and produces superior path feasibility, safety, and efficiency. These findings validate the effectiveness of the proposed algorithm.

1. Introduction

Path planning is a fundamental component in domains such as robotic navigation, autonomous driving, and networked intelligent systems, serving as a cornerstone for autonomous decision-making, obstacle avoidance, and efficient movement. Effective path planning not only significantly enhances the performance of robotic and autonomous systems but also optimizes travel time and energy

* Corresponding author.

Email addresses: xuzm@gxu.edu.cn (Z. Xu), Eric9527@st.gxu.edu.cn (G. Pang), laiyouyong@st.gxu.edu.cn (Y. Lai), huangyugan@st.gxu.edu.cn (Y. Huang), jianleizhang@nankai.edu.cn (J. Zhang).

<https://doi.org/10.1016/j.ins.2026.123586>

Received 3 September 2025; Received in revised form 15 April 2026; Accepted 3 May 2026

Available online 5 May 2026

0020-0255/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

consumption, and supports reliable task execution in intelligent IoT-oriented environments involving mobile agents and collaborative interaction [1]. Meanwhile, as intelligent systems become increasingly interconnected, issues related to secure information exchange have also attracted growing attention, further highlighting the practical significance of robust autonomous decision-making mechanisms [2]. As these technologies find applications in an increasing variety of complex and dynamic environments, the challenge of achieving efficient path planning becomes even more pronounced [3,4].

The core objective of path planning is to efficiently generate an optimal and collision-free path from a starting point to a target destination [5]. This paper specifically focuses on local path planning in contexts characterized by dynamic and incomplete information. In contrast to global path planning, local path planning operates with limited environmental information, necessitating that the agent actively observe nearby obstacles to determine viable actions, and explore visible areas to avoid local optima that could hinder exploration. Moreover, the agent must swiftly adapt to dynamic changes in the environment, optimizing the path within a constrained perception range to minimize path length and improve target efficiency. These challenges require robust exploration and planning capabilities to identify an optimal path with limited information.

Reinforcement Learning (RL) algorithms are particularly well-suited for path planning in complex, dynamic, and unknown environments because they do not rely on predefined rules, heuristic functions, or prior knowledge. Instead, RL algorithms continually refine strategies through experiential learning, enabling robots to autonomously make decisions and construct effective paths [6–8]. Despite these advantages, traditional RL approaches face limitations, notably the time-intensive training required for complex, high-dimensional environments that demand extensive exploration and trial and error [9]. Additionally, the choice of action-selection strategy is critical to path planning performance. Effective strategies must strike a balance between exploration—integral for learning environmental dynamics—and exploitation, which leverages learned knowledge for optimized decision-making. Overemphasis on exploration can lead to inefficiency, while premature exploitation risks convergence to suboptimal solutions. Finally, RL algorithms are heavily dependent on the design of the reward function, a pivotal element that directly affects the algorithms' effectiveness and efficiency. An inadequately designed reward function can cause the agent to adopt suboptimal strategies or become ensnared in local optima, thereby undermining the overall learning process.

Recent studies further suggest that, although DRL has shown strong potential for path planning, several challenges remain in dynamic and partially observable environments, including low sample efficiency, sparse or weak reward signals, and limited adaptability to environmental changes [10]. To address these issues, recent methods have explored prioritized experience replay, improved state representation, reward shaping, and planning-guided learning. For instance, Gök incorporated prioritized experience replay into D3QN for dynamic path planning; Tao and Kim combined Soft Actor-Critic (SAC) with the Dynamic Window Approach (DWA), tile coding, and a dense reward for local planning in dynamic environments; and Zhang et al. improved DRL-based path planning in complex environments through enhanced replay, heuristic reward design, and richer perception modeling [11–13]. These advances further underscore the importance of improving early-stage learning efficiency, action selection, and reward guidance, which together motivate the proposed A*-IDDQN framework.

To tackle the challenges of local path planning, this paper proposes A*-IDDQN, a method that integrates an improved Double Deep Q-Network (DDQN) with the A* algorithm. In contrast to prior A*-RL hybrid methods that incorporate A* into either the decision-making loop or the reward function, this work proposes a novel perspective: leveraging A* purely as an *offline initialization accelerator*. This redefinition offers two critical advantages. First, by explicitly leveraging A*'s deterministic global optimality, our method mitigates inefficient exploration during early training stages – a limitation prevalent in existing loosely integrated approaches. Second, compared to stochastic planners such as RRT* or PRM, A*'s optimality guarantees stable, high-quality initialization trajectories, thereby avoiding suboptimal pre-training that impedes Deep Reinforcement Learning (DRL) convergence. To address the high time cost during the initial exploration phase of training, the experience replay buffer of DDQN is pre-populated with paths generated by the A* algorithm. Additionally, randomness is introduced into the A* heuristic function to diversify the generated paths, providing varied experiences and preventing the agent from overfitting to a single path.

Standard DQN algorithms typically use an ϵ -greedy strategy for action selection. In this study, we enhance the agent's adaptability by incorporating an action selection strategy that dynamically adjusts based on the training stage and agent state, promoting a better exploration–exploitation balance without over-reliance on ϵ adjustments.

Conventional reward functions in path planning often fail to comprehensively account for safety, efficiency, and smoothness, leading to suboptimal path choices. A*-IDDQN improves on this by incorporating a more balanced reward structure: a collision penalty term and a goal-reaching reward term to enforce safety, an approach reward term to provide progressive guidance under limited perception, and a turning penalty term to encourage smoother trajectories. This design enables the agent to generate efficient and safe paths.

The proposed A*-IDDQN algorithm was evaluated through comparative experiments against several baseline methods, including the Original DQN, Double DQN, RRT-DDQN, and DM-DQN. The results demonstrate that A*-IDDQN achieves superior performance in terms of convergence speed, task success rate, and path quality, highlighting its potential and effectiveness for path planning applications. In addition, we incorporate classical planners, including RRT* and PRM, as non-learning baselines. Although these methods guarantee collision-free paths, their performance—particularly in terms of path smoothness and consistency—typically falls short of that achieved by A*-IDDQN.

The innovations of this paper are as follows:

- A* Initialization of the experience replay buffer: This study introduces a novel method to initialize the experience replay buffer of DDQN with heuristic paths generated by the A* algorithm. By leveraging A* to pre-generate feasible paths, this approach enhances the quality of the initial training data, addressing the common issue of slow convergence in the early stages of DDQN

training. The A*-derived paths provide a robust starting point for exploration, significantly improving the agent's ability to quickly adapt to the environment and accelerating the overall training process.

- A novel action selection strategy is introduced, which incorporates historical obstacle data within a unit-length radius around the agent. Unlike traditional approaches that solely prioritize actions with the highest Q-values, this strategy accounts for path safety, leading to more informed decision-making that minimizes collisions and reduces redundant actions. Additionally, a small probability of randomized action selection is maintained to preserve the agent's exploration capability, thereby enhancing its robustness in complex, dynamic environments.
- The proposed reward function integrates four components—target arrival reward, obstacle avoidance penalty, target proximity reward, and turning penalty. This multifaceted design ensures a well-rounded balance of safety, efficiency, and smoothness in path planning. Experimental results across a range of environments demonstrate the adaptability of this reward function to various map sizes and obstacle densities, enabling the agent to effectively learn optimal, collision-free paths even in challenging and dynamically changing scenarios.

The remainder of this paper is structured as follows: [Section 2](#) reviews the literature on solving path planning problems. [Section 3](#) describes the methods used in this study. [Section 4](#) introduces the experimental setup and presents the results. [Section 5](#) provides a discussion and analysis of the experimental results. Finally, [Section 6](#) concludes the paper.

2. Related work

To tackle the path planning problem in perception-constrained environments, a wide range of algorithms have been proposed, which can generally be divided into classical methods and RL-based approaches. Beyond the original formulations, many studies have sought to improve and optimize these algorithms, adapting them to more complex and dynamic scenarios.

2.1. Classic path planning methods

Dijkstra's algorithm, a well-known greedy technique for finding the shortest path in a graph [14], is effective in fully known environments. However, in perception-constrained environments, it must be augmented with additional strategies to handle limited local information. To enhance Dijkstra's applicability, Hart et al. introduced heuristic estimations to create the A* algorithm [15], which balances optimality, completeness, and flexibility. Despite these advantages, A* suffers from high memory consumption and often generates paths with excessive turns. Moreover, if the heuristic function is poorly defined, its efficiency may degrade to that of an exhaustive Dijkstra search.

The Rapidly-exploring Random Tree (RRT) algorithm, a sampling-based approach, is particularly suitable for path planning in unknown environments [16]. It constructs feasible paths by incrementally expanding a tree structure through random sampling, making it effective for high-dimensional and complex environments. However, due to its reliance on random sampling, RRT tends to produce paths with sharp turns and close proximity to obstacles, which may result in detours and inefficient trajectories.

2.2. Reinforcement learning

The experience replay mechanism plays a crucial role in RL, enabling the retention of past training experiences and the optimization of learning strategies. Li et al. enhanced the DQN experience replay by assigning sample weights and prioritizing training order, thereby preserving valuable data and eliminating redundant samples [7]. Yang et al. proposed a filtering method to retain positive experiences, thereby boosting experience reuse rates [17]. Additionally, Li et al. applied k-means clustering to ensure a diverse set of replay experiences [18]. Wei et al. introduced a quantum-inspired experience replay mechanism, where experiences were represented as qubits with importance determined by Temporal-Difference (TD) error, using quantum state amplitudes to maintain diversity among replayed experiences [19]. Cui et al. developed a Double Bias Experience Replay approach, which separately stores positive and negative samples to facilitate more effective learning of driving strategies [20].

The reward function in RL is essential, as it defines the problem in terms of accumulated rewards [21]. Liu et al. classified rewards into manually designed external rewards and parameterized internal rewards, proposing a deterministic policy gradient method to optimize collision avoidance and improve reward design [22]. Viswanadhapalli et al. employed a Lyapunov function-based reward shaping technique to accelerate training while ensuring policy convergence, thereby enhancing learning efficiency and stability [23]. To address the challenge of sparse rewards in drone navigation, Wang et al. developed a DRL algorithm that incorporates non-expert helpers (LwH), providing early exploration guidance that diminishes as the agent becomes more proficient [24].

Sharma et al. combined Q-matrix transfer learning with DQN to enhance action selection [25]. By initializing the agent with a pre-computed Q-matrix obtained from tabular Q-learning, they accelerated the learning process and reduced inefficient exploration. Liao et al. designed a hierarchical control framework for DQN in autonomous driving scenarios, which decouples driving and speed control decisions, leveraging state value and advantage function estimates to improve Q-table updates and enhance decision-making accuracy and stability [26]. Xu et al. introduced an action-decoupled Soft Actor-Critic (SAC) algorithm [27], enabling discrete and continuous action agents to learn and act independently, thereby optimizing performance in hybrid action spaces. Wang et al. proposed an action space compression method using the Yen algorithm to efficiently manage large action spaces by identifying the top K shortest paths, reducing action exploration costs and enhancing training efficiency by minimizing invalid actions [28].

Recent RL-based path planning studies indicate that replay efficiency, reward shaping, and representation quality remain key factors influencing performance. Gök integrated prioritized experience replay into D3QN and reported improvements in path length,

travel time, and collision avoidance in dynamic scenarios [11]. Tao and Kim designed a dense reward and combined SAC with tile coding and DWA to improve smoothness, goal-reaching efficiency, and adaptability in local dynamic planning [12]. Zhang et al. enhanced DRL-based path planning in complex environments through improved replay, heuristic reward design, and richer perception modeling, while Yadav et al. showed that optimized reward design can reduce both training iterations and trajectory length [13,29]. Overall, these studies highlight the continued importance of efficient experience utilization, informative reward design, and robust state representation in RL-based path planning.

2.3. Combination of classic algorithms and reinforcement learning

The integration of classic algorithms and RL effectively mitigates their respective limitations, enabling exploration while accelerating training convergence [30,31]. Heuristic rewards further assist agents in efficiently finding optimal solutions for path planning, especially in dynamic environments with uncertainty. Zhang et al. [32] integrated Particle Swarm Optimization (PSO) with RL to optimize paths and manage constraints, treating PSO particles as agents with tailored action selection to balance exploration and exploitation, thereby enhancing diversity and convergence. Cui et al. [33] improved the Artificial Bee Colony (ABC) algorithm by incorporating RL to adjust solution search updates, initializing Q-tables for each bee to avoid interference, which led to superior path length and execution time compared to traditional ABC. Yan et al. [34] addressed local optima issues in potential field methods by combining black hole potential fields with DQN, enabling agents to reach target points without prior knowledge. Xie's 2021 TSO algorithm [35], while efficient and featuring few control parameters, suffers from local optima in complex scenarios. To address this, Yan et al. [36] integrated TSO with Q-learning, allowing agents to select optimal strategies based on long-term rewards, thereby improving stability and search performance.

Compared to classical path planning alternatives – including both sampling-based algorithms (e.g., PRM, RRT*) and optimization-based techniques (e.g., PSO, ABC) – the A* algorithm is distinguished by its capacity to computationally generate structured, theoretically globally optimal paths via deterministic, heuristic-driven search. Conversely, stochastic sampling in PRM and RRT* often produces suboptimal trajectories with topological inconsistencies. Such deterministic characteristics render A* particularly compatible with RL frameworks, primarily as a source of high-fidelity prior experiences to accelerate learning during critical early-stage training.

Several studies have specifically utilized the A* algorithm to enhance RL-based path planning. Luo et al. [30] integrated A* with DQN by substituting the neural network-based action selection mechanism with A*-generated paths during the exploration phase. This approach accelerates decision-making and reduces the training time of the neural network. Zhang et al. [37] incorporated A*-derived path costs into the reward function for free-space navigation, further expediting the convergence of DQN. Yang et al. [38] took a hierarchical approach, decomposing the global path generated by A* into sub-goals and providing incremental rewards as the agent progresses toward each sub-goal. This strategy effectively mitigates the sparse reward problem commonly encountered in DRL.

Recent hybrid studies further demonstrate the effectiveness of combining classical planning mechanisms with DRL. Yang et al. proposed ST-D3QN, which uses A* to generate sub-target guidance and reward constraints for D3QN training, thereby improving convergence and path quality in UAV path planning [38]. Tao and Kim integrated SAC with the Dynamic Window Approach to enhance local path planning in the presence of dynamic obstacles [12]. Bi and Fang combined variable-direction potential fields with TD3 to address local-optimum and trap-obstacle problems in robot navigation [39]. These studies suggest that classical planning priors can be incorporated into RL through online guidance, motion-level control, or potential-field modulation. In contrast to these approaches, our method uses A*-generated paths solely as offline initialization samples for the replay buffer, thereby preserving a simple DDQN decision pipeline while improving early-stage training efficiency.

Distinct from conventional A*-RL integrations that embed A* within decision-making pipelines or utilize it for reward shaping, our method establishes a novel paradigm by employing A*-generated paths exclusively as offline initialization samples for the experience replay buffer. This framework delivers structurally coherent and information-rich experiences, enabling the DDQN agent to bypass unproductive exploration from the initial training stage. Consequently, it achieves substantial improvements in sample efficiency and accelerates early-phase convergence. To further enhance robustness and prevent overfitting to a single path, we introduce controlled stochasticity into the A* heuristic function, yielding a set of diverse but coherent initial trajectories.

The A* algorithm is renowned for its ability to generate globally optimal paths with high computational efficiency, leveraging heuristic search to guarantee both the feasibility of the shortest path and obstacle avoidance. However, in practical applications, A*-generated paths often fail to meet all operational requirements. For instance, these paths tend to exhibit non-smooth, zigzagging trajectories, which can degrade execution efficiency. Additionally, excessive path curvature may compromise the stability of movement and increase travel time, further limiting their practicality in dynamic environments.

While A* provides high-quality global path planning, its adaptability in complex environments remains limited, often leading to suboptimal local solutions and non-smooth trajectories. To overcome these limitations and improve the flexibility and real-time performance of path planning, we propose a hybrid approach that integrates A* with RL. RL facilitates dynamic strategy adjustments, addressing A*'s susceptibility to local optima and irregular paths. By combining the global planning strengths of A* with the local optimization capabilities of RL, our method generates paths that better meet multiple criteria, including smoothness, efficient obstacle avoidance, and enhanced real-time adaptability.

To substantiate the rationale behind selecting A* over other planners, we conduct a set of comparative experiments where PRM and RRT* are each used to generate initial experiences for DDQN, resulting in two variants: PRM-DDQN and RRT*-DDQN. These baselines were evaluated under identical conditions, with emphasis on early-stage metrics such as the success rate within the first 500 training episodes, the episode of the first successful collision-free path, path smoothness, and convergence stability. Results show that

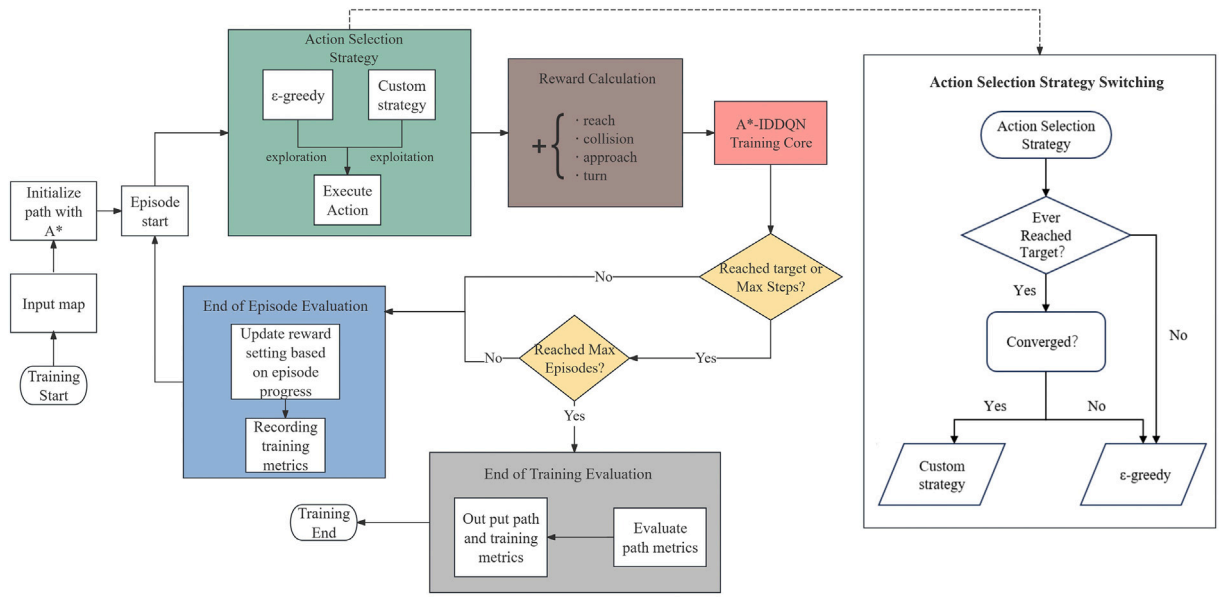


Fig. 1. Process of local path planning.

A*-IDDQN consistently outperforms both variants in terms of early success rate and path quality, affirming that A*'s deterministic and globally optimal trajectories provide more reliable guidance than the high-variance outputs of sampling-based methods. Full experimental results, including quantitative comparisons of training dynamics and trajectory efficiency, are detailed in Section 4.

3. Proposed method

The overall process of path planning based on local perception using the proposed A*-IDDQN algorithm is illustrated in Fig. 1.

Step 1: Training Start: Load the input map and initialize path planning using the A* algorithm. Populate the experience replay buffer with initial paths to provide prior path information for subsequent RL.

Step 2: Action Selection Strategy: Select the next action using an exploration or exploitation strategy. This includes the ϵ -greedy approach for exploration and a custom strategy for exploitation, followed by executing the chosen action. The custom strategy is activated only after the agent has reached the target successfully and the training process is verified to have converged. Prior to meeting these conditions, action selection follows the ϵ -greedy strategy. The specific convergence criteria are comprehensively detailed in Section 3.3.

Step 3: Reward Calculation: Calculate the reward based on the current state and executed action. Specifically, the reward function comprises four components, including the reward for reaching the target, the penalty for collisions or boundary violations, the reward for approaching the target, and the penalty for turning.

Step 4: A*-IDDQN Training Core: Update the A*-IDDQN Training Core through two processes: optimizing the online Q-network using the observed reward and state transition, and periodically synchronizing the target network with the online network at predefined episode intervals. The complete architecture of the Training Core is illustrated in Fig. 2.

Step 5: Goal Achievement or Maximum Step Reached: Check if the target has been reached or the maximum step count has been exceeded. If either condition is met, terminate the current episode and proceed to the next step; otherwise, return to Step 2.

Step 6: Maximum Episode Reached: Verify if the maximum number of episodes has been reached. If so, terminate training; otherwise, initiate a new episode.

Step 7: End of Episode Evaluation: Update the reward mechanism based on the episode's progress and record training metrics. Adjust the reward settings to align with the training progress and log metrics for evaluation.

Step 8: End of Training Evaluation: After all training episodes are completed, the final planned path is generated and the overall performance is evaluated using key path metrics, including path length, number of turns, and number of collisions.

The structure of the A*-IDDQN algorithm is illustrated in Fig. 2. The following sections will detail the standard DDQN, along with the enhancements made: initializing the experience replay buffer with the A* algorithm, refining the action selection strategy, and optimizing the reward function design.

3.1. DDQN

The standard DQN often suffers from Q-value overestimation, a common issue in RL. This refers to the tendency to overestimate certain state-action pairs during Q-value updates, which hinders the agent from learning an optimal policy. To mitigate this problem, Van et al. proposed the DDQN algorithm in 2016 [40].

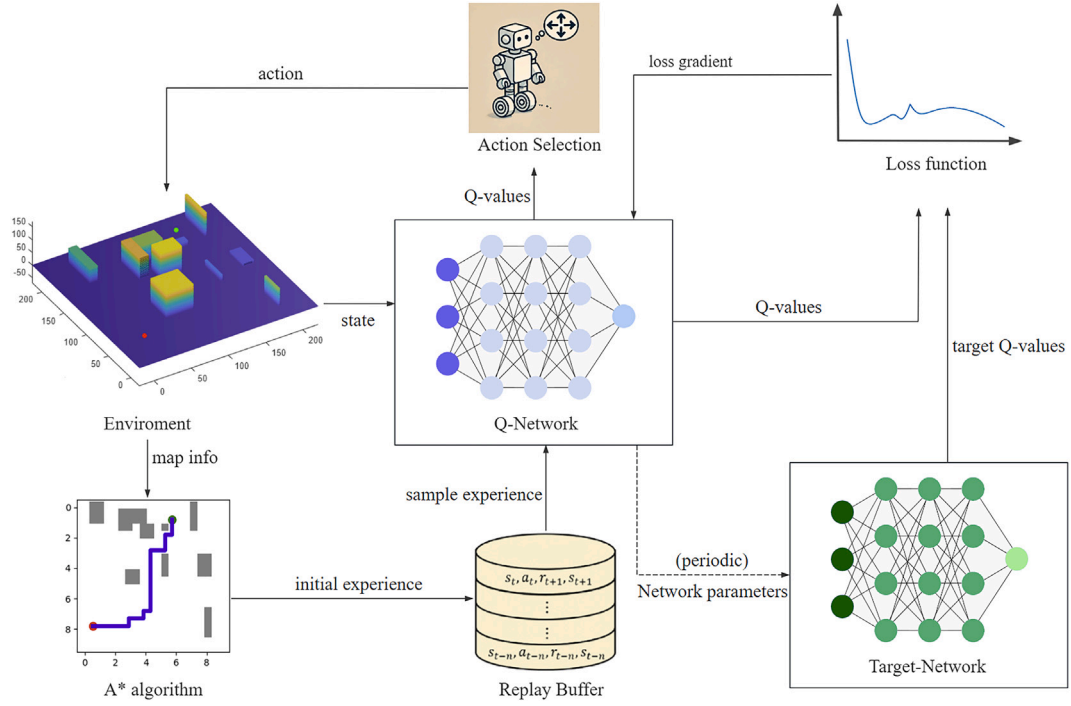


Fig. 2. Structure of A*-IDDQN.

DDQN differs from traditional Q-learning updates by employing two separate networks to decouple action selection from Q-value estimation: the Q Network, which selects actions, and the Target Network, which calculates the target Q-value for the chosen action.

In DDQN, the next action is first selected by the online Q-network according to

$$a' = \arg \max_a Q_{\text{online}}(s_{t+1}, a), \quad (1)$$

Eq. (1) indicates that the online Q-network selects the action a' that maximizes the estimated Q-value at state s_{t+1} . Here, Q_{online} denotes the trainable online network, which is responsible for greedy action selection in the DDQN framework.

The DDQN target is then computed as

$$y_t = r_t + \gamma Q_{\text{target}}(s_{t+1}, a'). \quad (2)$$

Here, $Q_{\text{target}}(s_{t+1}, a')$ denotes the target-network evaluation of action a' at the next state s_{t+1} . Specifically, the action is selected by the online Q-network using Eq. (1) and then evaluated by the target network. The remaining symbols have the same meanings as in the standard DQN.

To enhance the utilization of state-action transitions in RL, the experience replay mechanism was integrated into the DQN framework [19]. DQN stores the agent's past experiences—comprising state, action, reward, and next state—in a fixed-size experience pool, from which small batches are sampled for training. This approach effectively accelerates the processing of experiences and improves training efficiency.

3.2. A* algorithm

In the standard DDQN experience replay mechanism, samples in the experience buffer are typically generated through early random exploration, resulting in a significant discrepancy between these samples and the final optimal policy. Due to the lack of effective initial policy guidance, the model often exhibits low learning efficiency during the initial training stages. Additionally, the random sampling method does not prioritize high-quality experiences, which slows the model's convergence.

The A* algorithm can efficiently identify a minimum-cost path under a fixed cost function and heuristic setting [15]. Unlike other algorithms that generate paths randomly, A* is deterministic under a given heuristic function, yielding higher-quality paths and providing superior initial experience samples.

To address these challenges, this paper retrieves the map information required by the A* algorithm from the environment before training begins. Unlike the standard A* approach, the proposed method introduces randomness into the heuristic function, generating different paths with each execution of A*. This mechanism produces diverse initialization trajectories for DDQN, thereby improving early exploration quality and accelerating convergence in the initial training stage.

The A* algorithm combines the optimality guarantees of Dijkstra's algorithm with the efficiency of heuristic search. At each step, it selects the action that minimizes the function value $f(n)$, where the function $f(n)$ is determined by $g(n)$ and $h(n)$.

$$f(n) = g(n) + h(n), \quad (3)$$

where $g(n)$ represents the accumulated cost from the starting point to the current node, while $h(n)$ is the estimated cost from the current node to the goal, commonly referred to as the heuristic function. Since the complete path is not determined until the search is finished, the heuristic estimate may not always be entirely accurate.

The A* algorithm begins by initializing the open and closed lists, adding the start node to the open list, which is sorted by $f(n)$ values, while the closed list remains empty. The node with the lowest $f(n)$ value is selected as the current node. If this node is the target, the algorithm backtracks to construct the path; otherwise, it expands the unprocessed neighboring nodes, updates their $g(n)$ and $f(n)$ values, and adds them to the open list, adjusting parent nodes as needed. The current node is then moved to the closed list. This process repeats until the target is reached or the open list is empty, indicating either successful pathfinding or the absence of a feasible path.

In our implementation, the A* algorithm is employed without any trainable parameters or optimization processes. Its primary role is to generate initial trajectories that populate the experience replay buffer. To increase the diversity of the generated trajectories, we introduce stochastic perturbations to the heuristic function $h(n)$, allowing the algorithm to generate distinct yet feasible paths across successive executions. This randomized initialization procedure is repeated for a specified number of cycles, determined by the hyper-parameter *num_initializations*, ensuring that the buffer contains a diverse set of high-quality training samples. These initial experiences serve as valuable guidance during the early stages of DDQN training, significantly enhancing sample efficiency and facilitating faster convergence. This diversity improves the coverage of feasible local decision patterns in the replay buffer, reduces early over-concentration on a single path pattern, and helps stabilize learning during the initial training stage.

3.3. Action selection

The ϵ -greedy strategy is a widely used action selection method in RL [41]. Unlike a purely greedy approach that always chooses the action with the highest current Q-value, the parameter ϵ introduces exploration, enabling the agent to discover potentially better strategies. However, this method has certain limitations, such as the challenge of adjusting the balance between exploration and exploitation in response to dynamic environmental changes when using a fixed ϵ decay rate. Moreover, all actions are assigned equal selection probabilities during exploration, which can result in the execution of obviously suboptimal actions, thereby reducing learning efficiency.

To overcome these limitations, this paper proposes an improved action selection strategy that enhances the ϵ -greedy approach. The proposed method aims to more effectively utilize existing information during the exploitation phase, leading to better action selection.

3.3.1. Epsilon-greedy strategy

ϵ is a small decimal value between 0 and 1, used to balance exploration and exploitation. At each step, the agent selects the action with the highest current Q-value with a probability of $1-\epsilon$ (exploitation), while with a probability of ϵ (exploration), it randomly chooses an action from the action space A , which represents the set of all possible actions.

$$a = \begin{cases} \text{random action from } A, & \text{with probability } \epsilon \\ \arg \max_{a_t} Q(s, a), & \text{with probability } 1 - \epsilon. \end{cases} \quad (4)$$

In practical training, the value of ϵ is typically adjusted dynamically to balance exploration and exploitation as training progresses. A common approach is to gradually decay ϵ over time, enabling greater exploration during the early stages and increased exploitation of learned information to select optimal actions in the later stages.

3.3.2. Custom strategy

Unlike the standard DQN, which relies solely on the ϵ -greedy strategy for action selection, A*-IDDQN introduces an alternative action selection strategy specifically for the exploitation phase. This allows the agent to make more flexible decisions at different stages of training.

During operation, the agent records information about detected obstacles within a unit-length detection range around itself. The custom strategy leverages this historical data to make more informed and safer decisions, even if the selected action is not the one with the highest Q-value. To achieve this, a function is integrated to additionally account for path safety during action selection. This strategy reduces the likelihood of the agent choosing actions that lead to repeated paths or collisions with obstacles, making it more inclined to select safe actions.

When choosing actions, the agent primarily selects the safest action with the highest Q-value but also occasionally chooses an action randomly with a small probability. This ensures that the agent does not become overly reliant on existing paths during exploitation, retaining a degree of exploration. Even in scenarios where no safe actions are available, the agent can still randomly select actions to continue exploring the environment, thereby enhancing robustness in complex environments.

3.3.3. Strategy switching

The dynamic strategy-switching mechanism adaptively adjusts action selection based on the agent's path planning performance and training stability. Prior to each action selection, the system first verifies whether the agent has successfully reached the target at least once during training. If no successful completion has been recorded, the system employs the ϵ -greedy strategy to maintain exploration. Upon successful target attainment, the system evaluates the convergence status to determine the appropriate strategy: if convergence is confirmed, it switches to exploitation (i.e., the custom strategy); otherwise, the ϵ -greedy strategy remains active. The convergence assessment follows a rigorous four-stage verification framework:

1. **Training Sufficiency Verification:** Validate whether the current training episode count satisfies $episode \geq window + confirm_window$ to ensure statistical significance.

Algorithm 1 Custom strategy algorithm.

```

1: Input: current_position, path, epsilon = 0.01
2: Initialize possible_actions as a list of all possible actions
3: Initialize safe_actions as an empty list
4: for each action in possible_actions do
5:   new_position = current_position + move_corresponding_to(action)
6:   if is_valid_position(new_position) and not is_repeated_position(new_position, path) then
7:     Add action to safe_actions
8:   end if
9: end for
10: if safe_actions is not empty then
11:   if random number > epsilon then
12:     action = action_with_max_Q_value(safe_actions)
13:   else
14:     action = random_choice(safe_actions)
15:   end if
16: else
17:   action = random_choice(possible_actions)
18: end if
19: Return: action

```

Algorithm 2 Convergence check.

Require: reward sequence *rewards*, loss sequence *losses*, current episode *episode*, thresholds thr_r, thr_l , window size *window*, confirmation window size *confirm_window*

Ensure: **True** if converged, **False** otherwise

```

1: if episode < window + confirm_window then
2:   return False
3: end if
4: for i = window to episode - confirm_window do
5:    $var_r \leftarrow \text{Variance}(\text{rewards}[i - window : i])$ 
6:    $var_l \leftarrow \text{Variance}(\text{losses}[i - window : i])$ 
7:   if  $var_r < thr_r$  and  $var_l < thr_l$  then
8:     confirmed  $\leftarrow$  True
9:     for j = 1 to confirm_window do
10:       $vr \leftarrow \text{Variance}(\text{rewards}[i + j - window : i + j])$ 
11:       $vl \leftarrow \text{Variance}(\text{losses}[i + j - window : i + j])$ 
12:      if  $vr \geq thr_r$  or  $vl \geq thr_l$  then
13:        confirmed  $\leftarrow$  False
14:        break
15:      end if
16:    end for
17:    if confirmed then
18:      return True
19:    end if
20:  end if
21: end for
22: return False

```

2. **Stability Screening:** Implement a sliding window approach (with window size $window$) to analyze historical training records, computing the reward signal var_r and the loss function var_l within each window. Candidate convergence points are identified when both $var_r < thr_r \cap var_l < thr_l$ are satisfied.
3. **Stability Re-verification:** For each candidate convergence point, perform continuity validation over subsequent $confirm_window$ periods, requiring all consecutive verification windows to meet the stability criteria.
4. **Convergence State Determination:** If any candidate point successfully passes the consecutive validations, the model is certified as converged; otherwise, training is considered unconverged.

The complete convergence verification procedure is formally implemented in the pseudocode presented below.

3.4. Reward function

The reward function is one of the most critical components in RL. Traditional path planning algorithms often employ simple reward mechanisms, providing rewards for reaching the target and penalties for collisions. While such mechanisms are effective for basic tasks like obstacle avoidance and target acquisition, they have significant limitations.

First, without proper constraints, agents may generate longer or more convoluted paths to avoid obstacles. Second, providing positive rewards only upon reaching the target offers insufficient guidance throughout the process, leading to prolonged training times.

To address these issues, this paper designs a reward function that incorporates path safety, effectiveness, and smoothness, thereby enhancing overall path planning performance.

$$f_{\text{reward}} = f_{\text{reach}} + f_{\text{collision}} + f_{\text{approach}} + f_{\text{turn}}. \quad (5)$$

The reward function consists of four reward terms: f_{reach} , $f_{\text{collision}}$, f_{approach} , and f_{turn} . Specifically, f_{reach} provides a strong positive reward when the agent reaches the goal; $f_{\text{collision}}$ penalizes unsafe actions and obstacle collisions; f_{approach} rewards incremental progress toward the target; and f_{turn} penalizes unnecessary direction changes to encourage smoother paths. Together, these terms are designed to balance safety, efficiency, and smoothness in local path planning.

In a test scenario using a 20×20 map with 16 obstacles (each with an area of 5), representing a 20% obstacle coverage, experiments determined that the optimal $reach_reward$ is 250, while the $collision_penalty$ is 15. These parameters can be adapted for environments of varying sizes and obstacle densities.

Providing a substantial reward for reaching the target ensures that the agent remains focused on its primary objective, emphasizing the importance of reaching the target during the learning process.

The setting of f_{reach} is calculated using Eq. (6), where the agent's reward for reaching the target scales with the base success reward $reach_reward_{\text{base}}$ in the standard testing environment. The reward is adjusted according to the ratios of the current map_size and $num_obstacle$ to their standard values. As map_size or $num_obstacle$ increases, so does the difficulty, requiring a higher reward to maintain the target's attraction in complex environments.

$$f_{\text{reach}} = reach_reward_{\text{base}} \times \left(\frac{map_size}{map_size_{\text{base}}} \right) \times \left(\frac{num_obstacle}{num_obstacle_{\text{base}}} \right). \quad (6)$$

The agent must ensure not only target achievement but also path safety. By penalizing collision-prone actions, the agent learns to avoid hazardous areas, gradually identifying safer paths. Unlike approaches that reset the agent's position after collisions, the adaptive penalty method allows the agent to continue navigating towards the target, even if there are collision risks early in training. As training progresses, the agent refines its strategy, ultimately finding collision-free paths. This method improves exploration efficiency and enables the agent to autonomously discover safe paths without external intervention.

The penalty for colliding with obstacles is scaled based on the $collision_penalty_{\text{base}}$ in the standard testing environment. Specifically, both map_size_{base} and $collision_penalty_{\text{base}}$ are set to match the values used in the standard testing environment. The penalty decreases as the map_size increases, which prevents the agent from lingering in collision-free areas after collisions on larger maps. Conversely, the penalty increases with the $num_obstacle$, encouraging cautious behavior in more complex environments to enhance learning efficiency and task success.

$$f_{\text{collision}} = collision_penalty_{\text{base}} \times \left(\frac{map_size_{\text{base}}}{map_size} \right) \times \left(\frac{num_obstacle}{num_obstacle_{\text{base}}} \right). \quad (7)$$

The setting of f_{approach} follows Eqs. (8) and (9), guiding the agent towards the target point. Even without receiving f_{reach} for reaching the target, the agent can earn rewards for effective actions. $dist_{\text{current_to_target}}$ and $dist_{\text{original_to_target}}$ represent the current and previous distances of the agent to the target, respectively. Here, $position_{\text{current}}$ denotes the agent's position after executing the current action, $position_{\text{original}}$ refers to the position before the action, and $position_{\text{target}}$ represents the goal location. The quantity $dist_{\text{diff}}$ measures the progress made in a single step: it is positive when the agent moves closer to the target and non-positive otherwise. This metric serves to evaluate the effectiveness of each action in advancing toward the goal. If the agent's current distance to the target is less than the previous distance, indicating progress, the reward is calculated by multiplying $dist_{\text{diff}}$ by the scaling factor $map_size/10$, ensuring applicability across different map scales. No reward is given if the agent moves further from the target. The importance of f_{approach} lies in rewarding actions that bring the agent closer to the target, encouraging effective behavior. If no progress is made, no reward

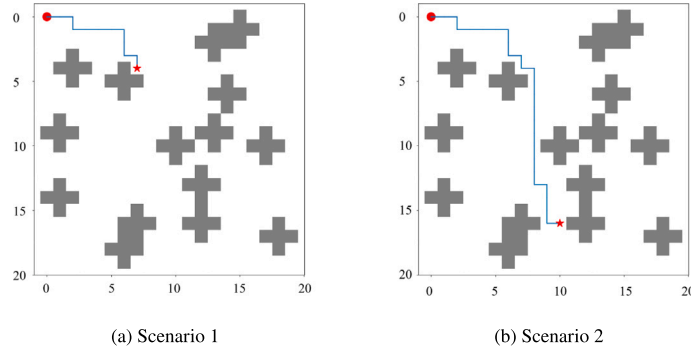


Fig. 3. An example to explain turn penalties.

is given, helping the agent find the path more efficiently.

$$\begin{cases} dist_{current_to_target} = \|position_{current} - position_{target}\| \\ dist_{original_to_target} = \|position_{original} - position_{target}\| \\ dist_{diff} = dist_{original_to_target} - dist_{current_to_target}, \end{cases} \quad (8)$$

$$f_{approach} = \begin{cases} dist_{diff} \times \frac{map_size}{10} & \text{if } dist_{diff} > 0 \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

The setting of f_{turn} is derived from Eqs. (10) and (11). The calculation process considers the number of obstacles surrounding the agent and the current training progress to dynamically adjust the turning penalty. This approach encourages the agent to minimize unnecessary turns while avoiding excessively high penalties during the early stages of training, which could otherwise hinder exploration.

$$penalty_{y_{init}} = \begin{cases} penalty_{min}, & \text{if } num_obstacle = 3 \\ penalty_{min} + 0.5 \times (penalty_{max} - penalty_{min}), & \text{if } num_obstacle = 2 \\ penalty_{min} + 0.75 \times (penalty_{max} - penalty_{min}), & \text{if } num_obstacle = 1 \\ penalty_{max}, & \text{if } num_obstacle = 0. \end{cases} \quad (10)$$

where $penalty_{min}$ is set to 0.1, and $penalty_{max}$ is set to 3.

As shown in Fig. 3, the agent's position during training is represented: the red circle marks the starting position, the blue line segment shows the current path, and the red star indicates the current position. In Fig. 3(a), with $num_obstacle = 2$, the calculated $penalty_{y_{init}}$ is 1.55. In Fig. 3(b), $num_obstacle = 1$ results in $penalty_{y_{init}} = 2.275$.

Eq. (10) calculates the initial turning penalty based on nearby obstacle information but does not account for the current training progress. Consequently, applying a constant turning penalty throughout training may limit the agent's ability to adapt its behavior over time, potentially resulting in suboptimal path planning or excessive penalties that restrict exploration in later training stages. To address this issue, Eq. (11) scales the penalty to an appropriate range, allowing for a gradual adjustment of the turning penalty during training, thus facilitating step-by-step optimization of the agent's behavior.

$$f_{turn} = penalty_{min} + (penalty_{y_{init}} - penalty_{min}) \times \frac{episode}{max_episode}. \quad (11)$$

3.5. Notation and hyper-parameters summary

See Table 1.

4. Experiments

This section presents the experimental setup, parameter settings, and environment configurations. We conduct extensive experiments to evaluate the path-planning performance of the proposed A*-IDDQN framework. The evaluation encompasses five distinct testing scenarios designed to systematically compare A*-IDDQN against several state-of-the-art RL baselines: Original DQN, Double DQN, RRT-DDQN [37], and DM-DQN [42]. To further demonstrate the superiority of A*-IDDQN over conventional non-learning approaches, we include comparative analyses with two classic path planning algorithms, RRT* and PRM, under identical testing scenarios. These five scenarios also serve as the foundation for our ablation studies examining the individual components of A*-IDDQN.

Table 1
Summary of notation and hyper-parameters in A*-IDDQN.

Symbol/Hyper-parameter	Description	Section	Default value/Range
$num_{initializations}$	Number of A* executions used to generate initialization trajectories for pre-populating the replay buffer	Section 3.2	20
$window$	Sliding-window size used to compute reward/loss variance during convergence checking	Section 3.3.3, Algorithm 2	100
$confirm_window$	Number of subsequent windows used for re-verifying convergence	Section 3.3.3, Algorithm 2	10
$thr_{(threshold_reward)}$	Variance threshold for reward stability in convergence checking	Section 3.3.3, Algorithm 2	40
$thr_{(threshold_loss)}$	Variance threshold for loss stability in convergence checking	Section 3.3.3, Algorithm 2	1
ϵ	Exploration rate in the ϵ -greedy policy, linearly decayed during training	Sections 3.3.1 and 4.1	$1.0 \rightarrow 0.1$
$target_update_frequency$	Frequency of synchronizing the target network with the online Q-network	Section 4.1	10
$batch_size$	Number of sampled transitions used in each replay-based update	Section 4.1	64
γ	Discount factor for future rewards in DDQN training	Sections 3.1 and 4.1	0.99
$max_episode$	Maximum number of training episodes	Section 4.1	1500
max_step	Maximum number of interaction steps per episode	Section 4.1	350

Table 2
Experimental parameters.

	Parameter name	Value	Description
Training hyperparameters	$num_initializations$	20	Number of times A* is used to accumulate experience
	$window$	100	Sliding window size for averaging rewards
	$confirm_window$	10	Confirmation window size for verifying convergence criteria
	$threshold_reward$	40	Convergence threshold for reward
	$threshold_loss$	1	Convergence threshold for loss
	$epsilon$	(0.1, 1)	Initial and final values for the ϵ -greedy strategy
	$target_update_frequency$	10	Frequency of target network updates
	$learning_rate$	0.001	Learning rate
	$batch_size$	64	Batch size for training updates from experience replay
	$gamma$	0.99	Discount factor for future rewards
Agent parameters	$num_validations$	100	Number of validation runs per experiment
	$action_dim$	4	Number of possible actions the agent can take
	$memory_size$	10,000	Experience replay buffer size using deque
Environment parameters	$max_episode$	1500	Maximum number of training episodes
	max_step	350	Maximum steps per episode

To provide a clearer evaluation perspective, the reported metrics are organized according to three desired path-planning criteria: safety, efficiency, and smoothness. Specifically, collision count and success rate are used to reflect safety and task reliability; path length and repetition count are used to characterize planning efficiency; the number of turns is used to evaluate path smoothness; and reward variance is used to indicate training stability.

4.1. Experiment settings

The map_size is set to 20, with $num_obstacle$ set to 16, resulting in an obstacle coverage of 20%. The agent's initial position is at [0,0], the top-left corner of the map, while the target point is at [19,19], the bottom-right corner. The agent occupies a single cell, with a detection radius also set to 1 cell. The specific experimental parameters are categorized into three classes and presented in Table 2.

In the ablation experiments, three key components of A*-IDDQN were tested individually: (1) without using A* for experience initialization, (2) without employing the custom action selection strategy, and (3) using a simplified reward setting (retaining only the basic reward for reaching the target point and the penalty for collisions).

The learning rate is set at 1×10^{-3} , with Adam as the optimizer. An ϵ -greedy strategy is used to balance exploration and exploitation, starting with $\epsilon = 1$ and linearly decreasing to 0.1. Training runs for up to 1500 episodes, with a maximum of 350 steps per episode. The discount factor γ is 0.99, and the batch size for experience replay is 64.

The experimental parameters were selected to match the scale and difficulty of the navigation task. The environment is configured as a 20×20 grid map with an obstacle coverage of 20%. With the start and target located at opposite corners, the theoretical minimum Manhattan distance is 38 steps. Considering the detours and stochastic exploration induced by limited perception and dynamic obstacles, the maximum number of steps per episode is set to 350. This value, which is approximately nine times the shortest path length, provides sufficient exploration space while preventing excessively inefficient trajectories.

For the DRL framework, the replay buffer is initialized using 20 independent A* runs. This amount of prior experience provides sufficiently diverse and informative transitions for early training, while reducing the risk of over-reliance on handcrafted trajectories. A replay memory size of 10,000 and a batch size of 64 are adopted to balance sample diversity and update stability. The discount factor γ is set to 0.99 to emphasize long-term return. In addition, the target network is synchronized every 10 episodes to improve training

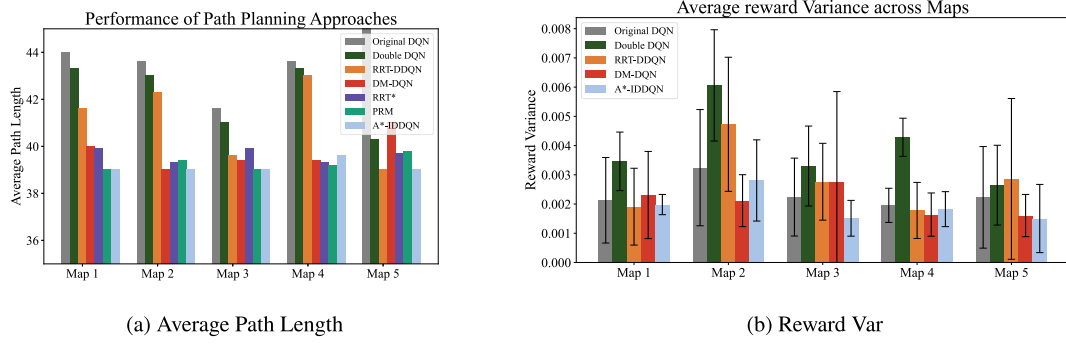


Fig. 4. Average path length and reward variance.

stability and reduce Q-value overestimation. The convergence-checking parameters ($window_size = 100$, $confirmation_window = 10$) are chosen to avoid declaring convergence based on short-term fluctuations.

The convergence thresholds, $threshold_reward = 40$ and $threshold_loss = 1$, were determined empirically with reference to the reward scale and loss magnitude observed during training. Given that the base reward for reaching the target is 250, a reward variance threshold of 40 indicates that the return has become relatively stable across consecutive episodes. Similarly, the loss threshold of 1 suggests that the Q-network updates have entered a relatively stable regime. Together, these thresholds help reduce the risk of prematurely identifying convergence before a robust policy has been formed.

The experience replay buffer is initialized with experiences from 20 runs of the A* algorithm, stored in a deque with a memory size of 10,000. During training, map instances and obstacle layouts are randomly generated under the same environmental settings. Initial exploration is conducted to fill the experience replay buffer before training begins. Batch data is then sampled from the buffer, and model updates occur every 10 episodes.

An episode ends and a new one begins if: (1) the maximum step limit is reached, or (2) the agent reaches the target. Unless otherwise stated, all learning-based baselines are trained and evaluated under a unified setup, including the same environment configuration, training episode budget, maximum number of steps per episode, and evaluation protocol.

It is worth noting that RRT* and PRM are classical sampling-based planning algorithms that operate without any learning mechanism. Consequently, these methods are excluded from performance evaluations based on reward accumulation and loss convergence metrics, which are specifically relevant to learning-based approaches. In contrast, such quantitative measures are integral to methods involving neural network training, policy optimization, and value function approximation, where these components are essential to the overall performance.

To rigorously evaluate the impact of initialization strategies on early-stage training performance, we implemented comparative experiments integrating PRM and RRT* with DDQN – denoted as PRM-DDQN and RRT*-DDQN, respectively. These studies focus on performance indicators critical to the initial learning phase (within the first 500 episodes), including: (1) the success rate of collision-free path planning, (2) the episode at which the first successful, collision-free path is achieved (first-reach episode), (3) the trajectory characteristics at episode 500, and (4) the loss evolution curves across early training. These metrics collectively offer a comprehensive evaluation of the efficiency and stability of different experience initialization strategies during the critical early learning stages of DRL-based planning.

Definition of event-based metrics: A successful path is defined as a trajectory that reaches the target without any collisions; the collision count is defined as the number of collision events with obstacles or map boundaries during one episode; the repetition count is computed as the path length minus the number of unique visited positions; and the turn count is defined as the number of direction changes between two consecutive moves.

4.2. Comparative experiment results

From the bar chart in Fig. 4(a), it can be observed that A*-IDDQN consistently plans shorter paths across various environments compared to DM-DQN, RRT-DDQN, Double DQN, and Original DQN. In Map 1 and Map 3, A*-IDDQN demonstrates a significant advantage in path length, clearly outperforming the other algorithms. However, in Map 5, the path length of A*-IDDQN is comparable to that of DM-DQN, and in Map 4, while A*-IDDQN's path performance surpasses most algorithms, it does not achieve the shortest path found by DM-DQN. Overall, A*-IDDQN still performs relatively well across different maps, successfully finding optimal paths in most environments.

Notably, A*-IDDQN demonstrates superior or comparable path length optimization relative to conventional sampling-based approaches (RRT* and PRM). While RRT* and PRM occasionally achieve comparable performance to A*-IDDQN (particularly in Maps 1-3), they consistently display higher performance variability and suboptimal robustness across diverse scenarios.

The data for Fig. 4(b), which presents a box plot of reward variance for different algorithms across various maps, is derived from the final 100 episodes of training and provides insight into the stability of the algorithms. The figure shows that A*-IDDQN maintains stable rewards in the later stages of training, outperforming other algorithms across multiple environments. In contrast, Double DQN and RRT-DDQN exhibit relatively high reward variances, indicating significant performance fluctuations in different environments.

Table 3
Average success rate of different algorithms.

Algorithm	Original DQN	Double DQN	RRT-DDQN	DM-DQN	RRT*	PRM	A*-IDDQN
Success rate	63.2%	68.3%	85.4%	83.9%	83.1%	100%	99.8%

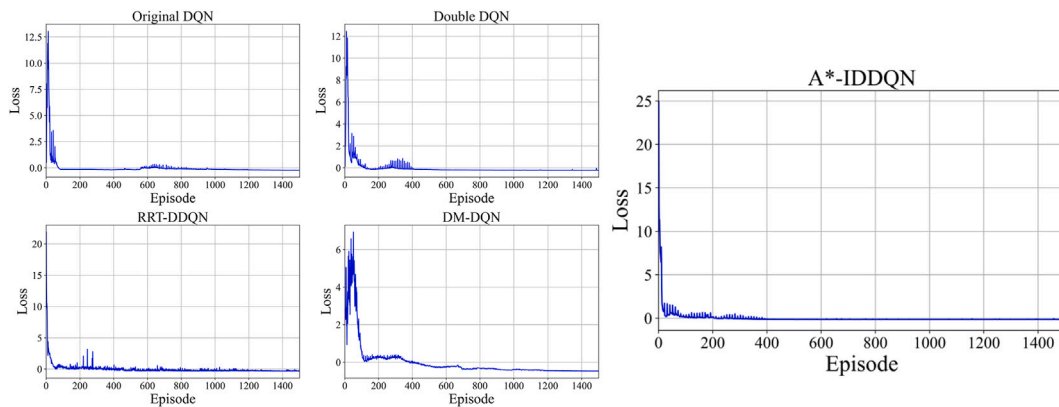


Fig. 5. Loss vs episode in training.

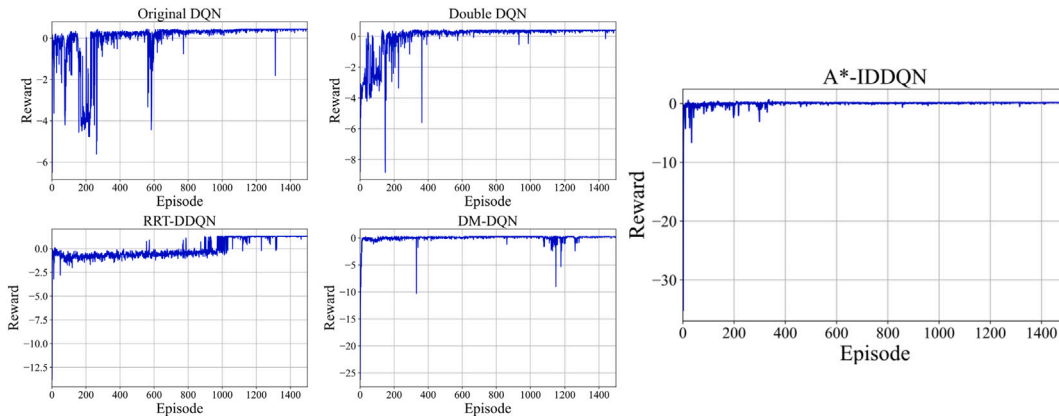


Fig. 6. Reward vs episode in training.

However, evaluating algorithm performance solely based on the average path length and reward variance is not comprehensive. According to the success rate data in Table 3, the success rates of Original DQN and Double DQN are significantly lower than that of A*-IDDQN (63.2% and 68.3% compared to 99.8%), indicating a higher likelihood of failure for Original DQN and Double DQN after multiple attempts. In contrast, A*-IDDQN achieves nearly 100% success rates across all environments, providing efficient and stable path planning results. Therefore, even though the path lengths in certain environments are similar to those produced by RRT-DDQN and DM-DQN, A*-IDDQN demonstrates superior robustness and reliability in path planning through significantly higher success rates.

Additionally, Fig. 5 presents the loss curves for each algorithm, showing that A*-IDDQN converges the fastest, with its loss rapidly decreasing and stabilizing in the early stages, indicating superior model convergence and stability. In contrast, the loss curves for Double DQN and RRT-DDQN exhibit significant fluctuations, particularly during the initial training phases, suggesting slower convergence and reduced stability.

In the reward curve depicted in Fig. 6, the average reward for A*-IDDQN increases rapidly and stabilizes, whereas the reward curves for Original DQN and Double DQN exhibit significant fluctuations, indicating slower adaptation to the environment and lower, less stable reward values. Notably, Original DQN shows frequent oscillations during the early stages of training, reflecting weaker adaptability. Compared to the other algorithms, A*-IDDQN reaches a stable reward phase more quickly and with considerably smaller fluctuations.

Moreover, the loss and reward values presented in this paper are normalized to standardize the ranges across different algorithms, facilitating meaningful comparisons. Notably, for RRT-DDQN and DM-DQN, which are based on prior works, the reward calculations adhere to the original settings, including varying success and collision reward values. Consequently, their reward settings differ from those of Original DQN, Double DQN, and A*-IDDQN. Direct comparisons of reward or loss values without normalization could be misleading. Normalized curves provide a unified scale that better reflects each algorithm's performance, enabling fairer comparisons.

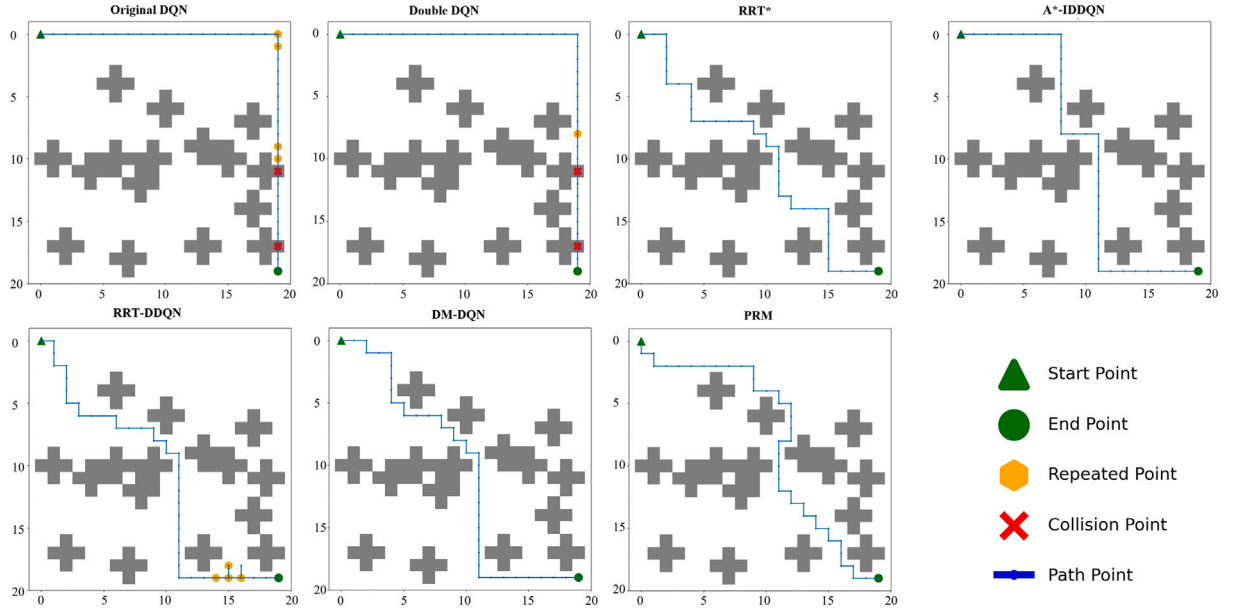


Fig. 7. Paths in Map 4.

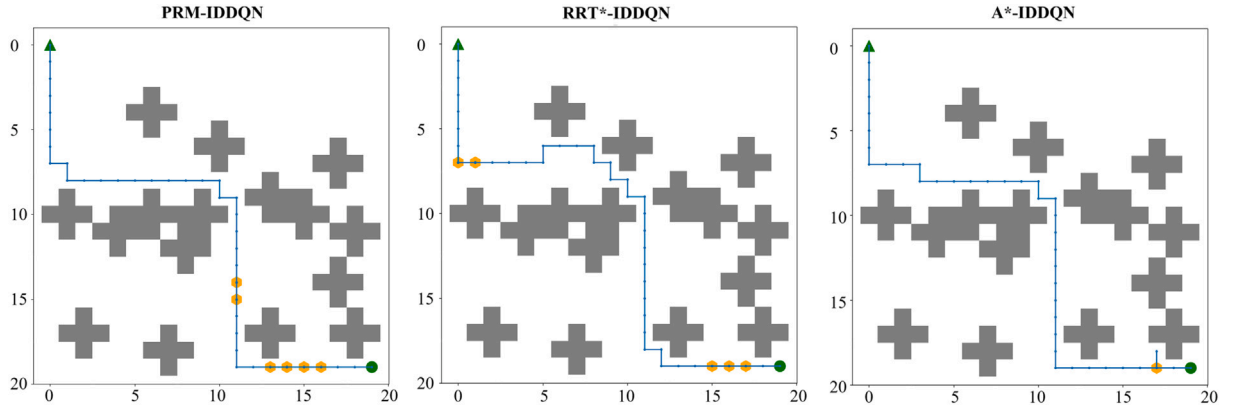


Fig. 8. Paths at episode 500 generated by PRM-IDDQN, RRT*-IDDQN, and A*-IDDQN.

Specifically, the Z-score normalization formula is used:

$$X' = \frac{X - \mu}{\sigma}. \quad (12)$$

In this context, X' represents the normalized value, X is the original value, μ denotes the mean of the original data, and σ is the standard deviation. This normalization method scales rewards and loss values uniformly across different algorithms, eliminating discrepancies due to varied reward settings and ensuring more consistent and scientifically valid comparisons.

Fig. 7 illustrates the paths generated by each algorithm at the end of a training session. The gray areas represent obstacles, the green triangle and circle denote the starting and target points of the path, respectively, the yellow hexagons mark coordinates where the path repeats, the red crosses indicate coordinates where the agent collides with obstacles, and the blue line segments represent the path points. As shown in Fig. 7, A*-IDDQN produces a more direct and coherent route around obstacles, with no repeated coordinates or collision markers in this scenario, indicating reduced backtracking and lower collision risk. This qualitative observation is consistent with the quantitative results in Table 7 for Map 4, where A*-IDDQN achieves 0 repetitions, 0 collisions, and only 4 turns, whereas several baseline methods exhibit higher repetition, collision, or turning counts.

As evidenced in Fig. 8, the paths generated by A*-IDDQN at episode 500 are markedly more structured, with significantly fewer redundant turns compared to PRM-IDDQN and RRT*-IDDQN. This suggests that A*-based initialization provides more effective early-stage guidance, yielding smoother and more efficient paths.

Quantitative comparisons are summarized in Tables 4 and 5. A*-IDDQN achieves both a higher cumulative success rate and an earlier onset of success, reinforcing its advantage in accelerating policy convergence and enhancing learning effectiveness in the early phase.

Table 4

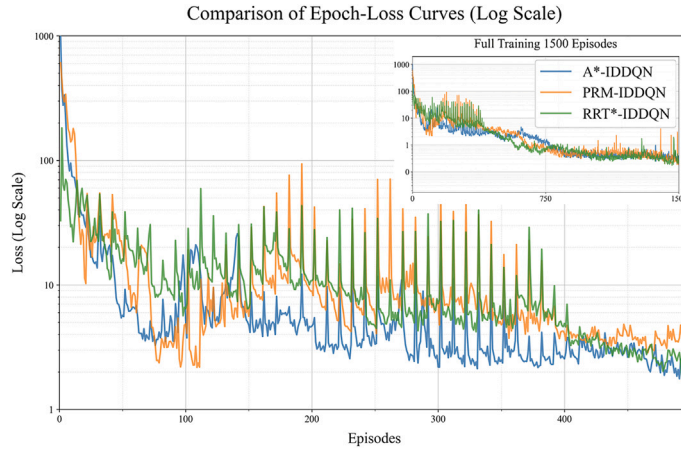
Early-stage success rate within the first 500 episodes.

Algorithm	PRM-IDDQN	RRT*-IDDQN	A*-IDDQN
Success rate (0–500 Episodes)	25.8%	21.6%	32.8%

Table 5

First successful collision-free arrival episode.

Algorithm	PRM-IDDQN	RRT*-IDDQN	A*-IDDQN
First reach episode	149	240	98

**Fig. 9.** Log-scale training losses within the first 500 episodes (Inset: full 1500 episodes).

Furthermore, Fig. 9 presents the log-scale training loss curves over the first 500 episodes, with an inset showing extended convergence trends up to 1500 episodes. The results clearly indicate that A*-IDDQN exhibits faster and more stable loss reduction than its PRM- and RRT*-based counterparts. These results validate our hypothesis that A*'s globally optimal, low-variance trajectories serve as more reliable initial experiences, leading to more efficient and stable learning in dynamic, partially known environments.

4.3. Ablation experiment results

In addition to the metrics reported in Section 4.2, the ablation study further examines the contributions of individual modules using two additional indicators: the episode of the first successfully collision-free path and the training time.

From the data presented in Table 6, it is evident that the A*-IDDQN algorithm outperforms others across most metrics in the various map tests, particularly showing more optimized results in terms of path length, collision count, and number of turns. Additionally, the number of episodes required to first reach the target and the overall training time are generally shorter than those of other ablation schemes, indicating that the A*-IDDQN algorithm has significant advantages in learning efficiency and convergence speed. Overall, the ablation results underscore the importance of A* initialization, the custom action selection strategy, and the complete reward function in enhancing algorithm performance.

5. Discussion

5.1. Comparative experiments

In the comparative experiments, each RL algorithm was executed five times on each map under identical settings of `max_episodes` and `max_steps`. Table 7 presents the results from a specific run, highlighting the consistent overall performance of each algorithm. As shown by the key metrics in Table 7, A*-IDDQN demonstrates significant advantages across various scenarios.

Firstly, regarding path length, A*-IDDQN consistently produces shorter paths across all scenarios. In each environment, A*-IDDQN achieves a path length of 39, representing the shortest route from start to target. In comparison, other RL algorithms generally exceed 40, particularly in Map 5, where A*-IDDQN maintains the shortest path, significantly outperforming Double DQN at 42 and Original DQN at 52. Traditional sampling-based planners, such as RRT* and PRM, exhibit competitive performance in terms of path length optimization, achieving identical path lengths of 39 in Maps 1–3, comparable to A*-IDDQN. However, performance discrepancies arise in more complex environments. Specifically, PRM produces a suboptimal path length of 41 in Map 4, while RRT* similarly achieves a path length of 41 in Map 5. In contrast, A*-IDDQN consistently maintains the optimal path length of 39 across all five test environments, highlighting its superior robustness and reliability when compared to both learning-based and sampling-based baseline methods.

Table 6
Ablation study key metrics.

	Metric	No A* Init	No custom actions	Simplified rewards	A*-IDDQN
Map 1	Path Length	39	42	43	39
	Repetitions	0	2	2	0
	Collisions	0	0	0	0
	Turns	3	7	7	3
	First Reach	18	12	10	6
	Training Time	05:06	06:45	04:04	03:26
Map 2	Path Length	39	41	39	39
	Repetitions	0	2	0	0
	Collisions	0	1	0	0
	Turns	8	9	22	6
	First Reach	34	10	8	8
	Training Time	07:19	03:53	04:05	03:08
Map 3	Path Length	39	50	43	39
	Repetitions	0	8	3	0
	Collisions	0	0	0	0
	Turns	2	16	25	2
	First Reach	20	35	16	13
	Training Time	13:05	04:55	03:30	03:28
Map 4	Path Length	39	47	39	39
	Repetitions	0	8	0	0
	Collisions	0	0	0	0
	Turns	6	12	18	4
	First Reach	148	213	92	111
	Training Time	04:17	04:16	05:14	03:40
Map 5	Path Length	39	45	39	39
	Repetitions	0	4	0	0
	Collisions	0	2	0	0
	Turns	8	13	12	6
	First Reach	14	289	264	42
	Training Time	07:13	08:16	03:34	04:02

Table 7
Algorithm comparison key metrics.

		Original DQN	Double DQN	RRT-DDQN	DM-DQN	RRT*	PRM	A*-IDDQN
Map 1	Path Length	46	44	41	41	39	39	39
	Repetitions	4	5	2	0	0	0	0
	Collisions	0	0	0	0	0	0	0
	Turns	12	9	7	22	11	27	3
Map 2	Path Length	45	42	43	39	39	39	39
	Repetitions	4	2	2	0	0	0	0
	Collisions	0	0	0	0	0	0	0
	Turns	15	13	25	17	20	31	6
Map 3	Path Length	43	39	39	39	39	39	39
	Repetitions	2	0	0	0	0	0	0
	Collisions	0	0	0	0	0	0	0
	Turns	11	2	21	15	16	31	2
Map 4	Path Length	43	40	47	39	39	41	39
	Repetitions	4	1	6	0	0	0	0
	Collisions	2	2	0	0	0	0	0
	Turns	5	3	23	14	14	23	4
Map 5	Path Length	52	42	39	41	41	39	39
	Repetitions	11	1	0	0	0	0	0
	Collisions	0	0	0	0	0	0	0
	Turns	21	12	10	16	17	18	6

Secondly, in terms of collisions, A*-IDDQN consistently maintains zero collisions across multiple runs, while Original DQN and Double DQN experience collisions to varying degrees. For instance, in Map 4, both Original DQN and Double DQN failed to find a collision-free path, whereas A*-IDDQN successfully avoided all collisions. This highlights the safety and stability of A*-IDDQN's path planning, effectively preventing collisions with obstacles. It should be emphasized that RRT* and PRM inherently ensure collision-free path planning by strictly operating within the obstacle-free configuration space.

Next, in terms of path repetition, A*-IDDQN recorded zero repetitions across all runs, indicating that the agent can accurately find a suitable path in a single planning attempt without requiring additional trials. Other algorithms showed higher repetition counts in certain scenarios; for example, in Map 5, the path planned by Original DQN had 11 repetitions, and RRT-DDQN had 6 repetitions in Map 4. In contrast, A*-IDDQN showed no repetitions in these scenarios, reflecting its high efficiency and reliability in path planning.

Lastly, with regard to the number of turns, A*-IDDQN consistently exhibits fewer turns than other algorithms, resulting in smoother paths with fewer directional changes—an important factor for practical applications. Across various scenarios, the number of turns for A*-IDDQN ranges from 2 to 6, whereas other algorithms reach up to 20 turns. The lowest turn count among the other algorithms is 2, achieved by Double DQN in Map 3, which only matches that of A*-IDDQN. This highlights the superior path quality of the A*-IDDQN algorithm, especially in complex environments where its advantages become more apparent. In contrast to learning-based approaches, RRT* and PRM typically generate paths with significantly higher turn counts, as their planning processes do not incorporate explicit smoothness optimization. Although these sampling-based methods guarantee collision avoidance, their resultant paths frequently exhibit non-optimal trajectories characterized by unnecessary directional changes and abrupt turns - a direct consequence of their stochastic sampling mechanisms and graph-based connection heuristics.

In summary, A*-IDDQN exhibits clear advantages across key metrics, including path length, collision count, repetition count, and number of turns, underscoring its superiority in path planning tasks. This indicates that A*-IDDQN not only finds shorter and more stable paths but also effectively avoids collisions and unnecessary path corrections, significantly enhancing the efficiency and accuracy of path planning.

Beyond comprehensive performance evaluation, we conduct targeted analysis on the early-stage behaviors of A*-IDDQN, PRM-DDQN, and RRT*-IDDQN. As illustrated in Fig. 8, A*-IDDQN produces significantly smoother and more direct paths within the first 500 training episodes. These early trajectories exhibit fewer turns and reduced repetition, suggesting more efficient policy initialization and faster adaptation to environmental constraints.

The quantitative metrics reported in Tables 4 and 5 further substantiate this observation. Specifically, A*-IDDQN achieves a 32.8% success rate in the first 500 episodes, outperforming PRM-DDQN (25.8%) and RRT*-IDDQN (21.6%). Moreover, A*-IDDQN reaches its first successful, collision-free target at episode 98, significantly earlier than PRM-DDQN (episode 149) and RRT*-IDDQN (episode 240). These results underscore the effectiveness of leveraging A* for initialization, which not only accelerates convergence but also promotes higher-quality early decision-making.

The loss curves in Fig. 9 provide additional insight. A*-IDDQN demonstrates a faster and more stable decrease in training loss, characterized by smoother gradients and lower fluctuations. In contrast, PRM-DDQN and RRT*-IDDQN exhibit noisy and inconsistent loss profiles, indicating unstable policy updates—likely due to the higher variance and suboptimal structure of their initial paths. This reinforces the notion that the benefits of A*-IDDQN stem not solely from reward shaping (e.g., turn penalties), but from structurally superior initialization data that facilitates more reliable learning dynamics.

5.2. Ablation experiments

In the ablation experiments, A*-IDDQN demonstrates significant advantages across various scenarios, as summarized in Table 6.

Firstly, in terms of path length, path repetition, and collision count, A*-IDDQN consistently generates the shortest, collision-free paths. It maintains minimal path lengths in all scenarios, with zero path repetitions, effectively avoiding unnecessary backtracking. A*-IDDQN also achieves zero collisions across all maps, highlighting its strong obstacle avoidance capabilities.

Secondly, regarding the number of turns, A*-IDDQN finds optimal solutions in most cases, matching the minimal turn count of the “No A* Initialization” scheme in Maps 1 and 3, while exhibiting fewer turns in other maps, resulting in smoother paths.

Thirdly, in episodes required to first reach the target, A*-IDDQN consistently finds the target sooner. For instance, in Map 1, A*-IDDQN reaches the target in episode 6, whereas other schemes achieve this in episode 10 or later. In Map 4, A*-IDDQN reaches the target in episode 111, significantly ahead of the “No Custom Action Selection Strategy” scheme, which does so in episode 213.

Finally, A*-IDDQN demonstrates superior training efficiency. In Maps 1 to 4, it has shorter training times, with Map 3 showing a stark contrast—03:28 for A*-IDDQN compared to 13:05 for the “No A* Initialization” scheme. In Map 5, A*-IDDQN is slightly behind the fastest scheme (04:02 vs. 03:34), still demonstrating near-optimal training efficiency even in complex environments.

Overall, A*-IDDQN excels in key metrics, including path optimization, early target achievement, and training time, demonstrating exceptional performance. The ablation experiments highlight the critical roles of A* initialization, custom action selection, and reward function design in enhancing A*-IDDQN, accelerating learning, and ensuring safe and accurate path planning across diverse environments.

For clarity, the main module-to-impact relationships can be summarized as follows:

- **A* initialization:** mainly improves early learning efficiency, as removing it generally delays the first successful arrival and increases training time on several maps.
- **Custom action selection:** mainly reduces path repetitions and collisions, while also helping to avoid unnecessary detours.
- **Complete reward design:** mainly reduces the number of turns and therefore improves path smoothness.

5.3. Result analysis

The outstanding performance of A*-IDDQN primarily stems from its core improvements over DQN.

Firstly, initializing the experience replay buffer is critical for A*-IDDQN's performance. Using diverse paths generated by the A* algorithm to populate the replay buffer provides the agent with rich, high-quality experiences from the outset, addressing the inefficiency of standard DQN during early random exploration and enabling faster strategy learning. By adding randomness to A*'s heuristic function, path diversity is ensured, preventing over-reliance on a single path and enhancing strategy generalization. Experimental results show that removing A* initialization significantly increases the number of episodes needed to first reach the target and extends training time, underscoring reduced efficiency in the early phases without initial experience support. This highlights the importance of A* initialization in helping the agent quickly adapt to larger and more complex environments, thus improving strategy generalization.

In addition to improvements over standard DQN, comparative results further validate the effectiveness of the A*-based initialization strategy. A*-IDDQN consistently outperforms PRM-IDDQN and RRT*-IDDQN in early-stage metrics, achieving both higher success rates and significantly earlier first successful, collision-free paths. These findings underscore the contribution of A*'s deterministic and globally optimal structure in providing stable and informative prior experiences, which in turn accelerate convergence and improve exploration efficiency in complex navigation scenarios.

Moreover, the early-stage paths generated by A*-IDDQN demonstrate higher quality in terms of smoothness and path compactness. Compared to PRM-IDDQN and RRT*-IDDQN, A*-IDDQN exhibits notably fewer redundant turns and repetitions within the first 500 episodes, leading to more stable policy learning and efficient spatial reasoning. These results emphasize that A* initialization not only improves convergence speed but also enhances the structural quality of early-stage decision-making.

Secondly, the enhanced action selection strategy in A*-IDDQN improves the agent's decision-making flexibility, addressing the limitations of the standard DQN's ϵ -greedy strategy in balancing exploration and exploitation. This adaptive strategy dynamically adjusts to the training phase and current state, allowing the agent to select optimal actions across different environments and tasks, thereby boosting learning efficiency and strategy quality. The removal of this custom strategy leads to unstable exploration, longer training times, and more episodes needed to reach the target, emphasizing the standard ϵ -greedy strategy's lack of dynamic adaptability. This underscores the critical role of the custom action selection strategy in effectively balancing exploration and exploitation for efficient learning.

The convergence behavior of each method further corroborates the effectiveness of our design choices. The loss curve of A*-IDDQN within the first 500 episodes shows a faster decline with fewer fluctuations compared to PRM-IDDQN and RRT*-IDDQN, as seen in Fig. 9. This smooth convergence reflects improved sample efficiency, which can be attributed to the synergy between A* initialization, adaptive action selection, and well-structured reward design.

Furthermore, the reward function design plays a pivotal role in A*-IDDQN's consistent performance across different environments. Using Eqs. (6) and (7), rewards for reaching the target and collision penalties are dynamically adjusted based on map size and obstacle density. In larger maps, lower collision penalties encourage broader exploration, while higher penalties in densely populated areas promote cautious behavior to minimize collisions. Rewards for approaching the target, defined in Eqs. (8) and (9), provide positive incentives before reaching the goal, effectively guiding the agent. The turning penalty, calculated by Eqs. (10) and (11), ensures smoother paths by discouraging unnecessary turns. Simplifying the reward function results in less smooth paths, increased collisions, and more frequent turns in complex settings, highlighting the role of the original reward function in balancing exploration and exploitation and guiding the agent to adapt to different environments, thus enhancing path planning efficiency and safety.

Compared to RRT-DDQN, A*-IDDQN's advantage lies in utilizing A*-generated paths as experience, rather than directly influencing the agent's strategy with RRT-generated paths, which can lead to learning instability. In contrast to DM-DQN, A*-IDDQN benefits from a more adaptable reward function, enhancing its performance across environments of varying complexity. The DM-DQN reward function, based on artificial potential fields, inadequately accounts for the impact of obstacle density on agent behavior, leading to suboptimal navigation in high-density areas and failing to balance target attraction and obstacle avoidance, thereby compromising path planning efficiency and safety.

In summary, A*-IDDQN significantly enhances agent performance in path planning through improvements in experience replay initialization, action selection strategy, and reward function design. These enhancements collectively enable the agent to learn high-quality strategies more efficiently, resulting in shorter, smoother, and collision-free paths, ultimately increasing task success rates and planning efficiency. Experimental results demonstrate the effectiveness of A*-IDDQN in path planning, offering a valuable reference for future research.

6. Conclusion

This paper proposes A*-IDDQN, an innovative path planning algorithm that synergistically integrates the A* algorithm with an enhanced DDQN, aiming to improve agent performance in complex and unknown environments. By enriching the experience replay buffer of the DQN with A*-generated paths, A*-IDDQN significantly enhances learning efficiency, particularly in the early stages of training. Furthermore, a customized action selection strategy increases the flexibility of decision-making, while an adaptive reward function is designed to guide the agent away from local optima. Experimental results demonstrate that A*-IDDQN consistently outperforms existing methods in planning success rate, training stability, path feasibility, safety, and path length across a variety of environments.

The loss curves indicate that A*-IDDQN achieves faster convergence and maintains a lower overall loss during training compared to alternative algorithms. Although all methods eventually stabilize, A*-IDDQN exhibits a more rapid decline in loss, reflecting superior learning efficiency facilitated by the incorporation of A* heuristic paths as structured initial experiences. In contrast, approaches such as RRT-DDQN and DDQN display greater fluctuations and delayed convergence during learning, suggesting less stable convergence and less efficient training dynamics.

The reward curves further highlight the advantages of A*-IDDQN. It achieves higher and more stable rewards more quickly compared to competing methods, converging rapidly to optimal reward values. This validates the efficacy of incorporating A* heuristics into the DDQN framework as an offline initialization strategy. In comparison, algorithms like Original DQN and RRT-DDQN suffer from more significant reward fluctuations, reflecting less stable decision-making throughout training.

Reward variance analysis provides additional evidence of A*-IDDQN's robustness. It maintains lower reward variance across different environments, demonstrating strong generalization and consistent performance. A*-IDDQN not only achieves high learning efficiency but also exhibits reliable behavior across diverse scenarios, confirming its superiority in path planning tasks.

In summary, A*-IDDQN significantly enhances agent performance in path planning through a novel combination of experience replay initialization, an adaptive action selection strategy, and an optimized reward function. These enhancements collectively enable the agent to learn high-quality strategies more efficiently, leading to shorter, smoother, and collision-free paths, ultimately improving task success rates and planning efficiency.

CRedit authorship contribution statement

Zimin Xu: Writing – review & editing, Resources, Project administration, Funding acquisition, Conceptualization. **Guangcheng Pang:** Writing – review & editing, Writing – original draft, Visualization, Methodology, Investigation, Data curation. **Youyong Lai:** Validation, Data curation. **Yugan Huang:** Writing – review & editing. **Jianlei Zhang:** Writing – review & editing, Supervision, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported in part by the [National Natural Science Foundation of China](#) under Grant 62473211.

Data availability

Data will be made available on request.

References

- [1] J. Guo, A. Liu, K. Ota, M. Dong, X. Deng, N.N. Xiong, ITCN: an intelligent trust collaboration network system in IoT, *IEEE Trans. Netw. Sci. Eng.* 9 (1) (2021) 203–218.
- [2] X. Chen, Z. Zhang, A. Qiu, Z. Xia, N.N. Xiong, Novel coverless steganography method based on image selection and StarGAN, *IEEE Trans. Netw. Sci. Eng.* 9 (1) (2020) 219–230.
- [3] T. Gu, J.M. Dolan, On-road motion planning for autonomous vehicles, in: *Intelligent Robotics and Applications: 5th International Conference, ICIRA 2012, Montreal, QC, Canada, October 3–5, 2012, Proceedings, Part III 5*, Springer, 2012, pp. 588–597.
- [4] B. Paden, M. Čáp, S.Z. Yong, D. Yershov, E. Frazzoli, A survey of motion planning and control techniques for self-driving urban vehicles, *IEEE Trans. Intell. Veh.* 1 (1) (2016) 33–55.
- [5] D. Agarwal, P.S. Bharti, Implementing modified swarm intelligence algorithm based on slime moulds for path planning and obstacle avoidance problem in mobile robots, *Appl. Soft Comput.* 107 (2021) 107372.
- [6] T. Zhang, H. Mo, Reinforcement learning for robot research: a comprehensive review and open issues, *Int. J. Adv. Robot. Syst.* 18 (3) (2021), <https://doi.org/10.1177/17298814211007305>
- [7] J. Li, Y. Chen, X. Zhao, J. Huang, An improved DQN path planning algorithm, *J. Supercomput.* 78 (1) (2022) 616–639.
- [8] Y. Yang, L. Juntao, P. Lingling, Multi-robot path planning based on a deep reinforcement learning DQN algorithm, *CAAI Trans. Intell. Technol.* 5 (3) (2020) 177–183.
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [10] L. Xu, W. Zhang, Survey on path planning based on deep reinforcement learning, in: *International Conference on Machine Learning and Intelligent Computing*, PMLR, 2025, pp. 685–695.
- [11] M. Gök, Dynamic path planning via dueling double deep Q-network (D3QN) with prioritized experience replay, *Appl. Soft Comput.* 158 (2024) 111503.
- [12] B. Tao, J.-H. Kim, Deep reinforcement learning-based local path planning in dynamic environments for mobile robot, *J. King Saud Univ.-Comput. Inf. Sci.* 36 (10) (2024) 102254.
- [13] Z. Zhang, H. Fu, J. Yang, Y. Lin, Deep reinforcement learning for path planning of autonomous mobile robots in complicated environments, *Complex Intell. Syst.* 11 (6) (2025) 277.
- [14] E.W. Dijkstra, A note on two problems in connexion with graphs, *Numer. Math.* 1 (1) (1959) 269–271.
- [15] P.E. Hart, N.J. Nilsson, B. Raphael, A formal basis for the heuristic determination of minimum cost paths, *IEEE Trans. Syst. Sci. Cybern.* 4 (2) (1968) 100–107.
- [16] S.M. LaValle, J.J. Kuffner, Rapidly-exploring random trees: progress and prospects, *Algorithmic Comput. Robot.* (2001) 303–307.
- [17] J. Yang, J. Ni, M. Xi, J. Wen, Y. Li, Intelligent path planning of underwater robot based on reinforcement learning, *IEEE Trans. Autom. Sci. Eng.* 20 (3) (2022) 1983–1996.
- [18] M. Li, T. Huang, W. Zhu, Clustering experience replay for the effective exploitation in reinforcement learning, *Pattern Recognit.* 131 (2022) 108875.
- [19] Q. Wei, H. Ma, C. Chen, D. Dong, Deep reinforcement learning with quantum-inspired experience replay, *IEEE Trans. Cybern.* 52 (9) (2021) 9326–9338.
- [20] J. Cui, L. Yuan, L. He, W. Xiao, T. Ran, J. Zhang, Multi-input autonomous driving based on deep reinforcement learning with double bias experience replay, *IEEE Sens. J.* 23 (11) (2023) 11253–11261.
- [21] D. Silver, S. Singh, D. Precup, R.S. Sutton, Reward is enough, *Artif. Intell.* 299 (2021) 103535.
- [22] Q. Liu, Z. Liu, B. Xiong, W. Xu, Y. Liu, Deep reinforcement learning-based safe interaction for industrial human-robot collaboration using intrinsic reward function, *Adv. Eng. Inform.* 49 (2021) 101360.
- [23] J.K. Viswanadhapalli, V.K. Elumalai, S. Shivram, S. Shah, D. Mahajan, Deep reinforcement learning with reward shaping for tracking control and vibration suppression of flexible link manipulator, *Appl. Soft Comput.* 152 (2024) 110756.

- [24] C. Wang, J. Wang, J. Wang, X. Zhang, Deep-reinforcement-learning-based autonomous UAV navigation with sparse rewards, *IEEE Internet Things J.* 7 (7) (2020) 6180–6190.
- [25] J. Sharma, P.-A. Andersen, O.-C. Granmo, M. Goodwin, Deep Q-learning with Q-matrix transfer learning for novel fire evacuation environment, *IEEE Trans. Syst. Man Cybern. Syst.* 51 (12) (2020) 7363–7381.
- [26] J. Liao, T. Liu, X. Tang, X. Mu, B. Huang, D. Cao, Decision-making strategy on highway for autonomous vehicles using deep reinforcement learning, *IEEE Access* 8 (2020) 177804–177814.
- [27] Y. Xu, Y. Wei, K. Jiang, L. Chen, D. Wang, H. Deng, Action decoupled SAC reinforcement learning with discrete-continuous hybrid action spaces, *Neurocomputing* 537 (2023) 141–151.
- [28] W. Wang, L. Tang, T. Liu, X. He, C. Liang, Q. Chen, Towards reliability-enhanced, delay-guaranteed dynamic network slicing: a multi-agent DQN approach with an action space reduction strategy, *IEEE Internet Things J.* 11 (6) (2024) 9282–9297.
- [29] A.K. Yadav, P. Sharma, X. Cheng, S.S.P. Shukla, Reinforcement learning-based intelligent path planning for optimal navigation in dynamic environments, *Neural Process. Lett.* 58 (2026) 10.
- [30] L. Luo, N. Zhao, Y. Zhu, Y. Sun, A* guiding DQN algorithm for automated guided vehicle pathfinding problem of robotic mobile fulfillment systems, *Comput. Ind. Eng.* 178 (2023) 109112.
- [31] R.A. Bianchi, M.F. Martins, C.H. Ribeiro, A.H. Costa, Heuristically-accelerated multiagent reinforcement learning, *IEEE Trans. Cybern.* 44 (2) (2013) 252–265.
- [32] X. Zhang, S. Xia, X. Li, T. Zhang, Multi-objective particle swarm optimization with multi-mode collaboration based on reinforcement learning for path planning of unmanned air vehicles, *Knowl.-based Syst.* 250 (2022) 109075.
- [33] Y. Cui, W. Hu, A. Rahmani, A reinforcement learning based artificial bee colony algorithm with application in robot path planning, *Expert Syst. Appl.* 203 (2022) 117389.
- [34] Q. Yao, Z. Zheng, L. Qi, H. Yuan, X. Guo, M. Zhao, Z. Liu, T. Yang, Path planning method with improved artificial potential field—a reinforcement learning perspective, *IEEE Access* 8 (2020) 135513–135523.
- [35] L. Xie, T. Han, H. Zhou, Z.-R. Zhang, B. Han, A. Tang, Tuna swarm optimization: a novel swarm-based metaheuristic algorithm for global optimization, *Comput. Intell. Neurosci.* 2021 (1) (2021) 9210050.
- [36] Z. Yan, J. Yan, Y. Wu, S. Cai, H. Wang, A novel reinforcement learning based tuna swarm optimization algorithm for autonomous underwater vehicle path planning, *Math. Comput. Simul.* 209 (2023) 55–86.
- [37] F. Zhang, C. Gu, F. Yang, An improved algorithm of robot path planning in complex environment based on double DQN, in: *Advances in Guidance, Navigation and Control: Proceedings of 2020 International Conference on Guidance, Navigation and Control, ICGNC 2020, Tianjin, China, October 23–25, 2020*, Springer, 2022, pp. 303–313.
- [38] A. Yang, J. Huan, Q. Wang, H. Yu, S. Gao, ST-D3QN: advancing UAV path planning with an enhanced deep reinforcement learning framework in ultra-low altitudes, *IEEE Access* 13 (2025) 65285–65300.
- [39] Y. Bi, X. Fang, A hybrid path planning framework integrating deep reinforcement learning and variable-direction potential fields, *Mathematics* 13 (14) (2025) 2312.
- [40] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, 2016.
- [41] P. Vamplew, R. Dazeley, C. Foale, Softmax exploration strategies for multiobjective reinforcement learning, *Neurocomputing* 263 (2017) 74–86.
- [42] Y. Gu, Z. Zhu, J. Lv, L. Shi, Z. Hou, S. Xu, DM-DQN: dueling munchausen deep Q network for robot path planning, *Complex Intell. Syst.* 9 (4) (2023) 4287–4300.